Traversal in Circular Doubly Linked List (CDLL)

What is Traversal?

Traversal means visiting each node of the linked list exactly once to access or display its elements.

In a **Circular Doubly Linked List**, traversal is special because:

- There is **no NULL**
- The list is **circular**
- We stop when we reach the **head again**

Types of Traversal

1. **Forward Traversal** (using `next`)
2. **Backward Traversal** (using `prev`)

Program in C

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};

/* Create CDLL (for testing) */
struct Node* insertEnd(struct Node *head, int value)
{
    struct Node *newnode, *last;

    newnode = (struct Node*)malloc(sizeof(struct Node));
    newnode->data = value;

    if(head == NULL)
    {
        newnode->next = newnode;
        newnode->prev = newnode;
        return newnode;
    }
```

```
    last = head->prev;

    newnode->next = head;
    newnode->prev = last;

    last->next = newnode;
    head->prev = newnode;

    return head;
}

/* Forward Traversal */
void traverseForward(struct Node *head)
{
    struct Node *temp;

    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = head;

    printf("Forward Traversal: ");
    do
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    while(temp != head);

    printf("\n");
}

/* Backward Traversal */
void traverseBackward(struct Node *head)
{
    struct Node *temp;

    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = head->prev; // start from last node

    printf("Backward Traversal: ");
```

```
    do
    {
        printf("%d ", temp->data);
        temp = temp->prev;
    }
    while(temp != head->prev);

    printf("\n");
}

/* Main Function */
int main()
{
    struct Node *head = NULL;

    // Creating list
    head = insertEnd(head, 10);
    head = insertEnd(head, 20);
    head = insertEnd(head, 30);

    // Traversals
    traverseForward(head);
    traverseBackward(head);

    return 0;
}
```

Explanation

Forward Traversal

- Start from `head`
- Move using `next` pointer
- Stop when `temp == head` again

Backward Traversal

- Start from `head->prev` (last node)
- Move using `prev` pointer
- Stop when reaching last node again

Sample Output

```
Forward Traversal: 10 20 30
Backward Traversal: 30 20 10
```